TTL (Time-To-Live) : 캐시 항목에 만료 시간을 두는 방식
Invalidation : 원본 데이터가 바뀌면 캐시 키를 삭제하거나 새 값으로 덮어쓰는 방식
Versioned key : 키에 버전을 넣어 데이터가 바뀌면 버전 올려 새 키를 쓰고, 예전 키는 TTL로 자연 소멸
Pub/Sub (Publish/Subscribe) 발행자가 메시지를 채널에 보내면, 구독 중인 모든 서버가 그 메시지를 받는 통신 방식

로컬 캐시
빠르지만 stale (신선하지 않은) 데이터를 가질 수 있음

Remote-Local 정합성
= 공유 캐시의 값과 각 서버의 로컬 메모리 캐시 값이 서로 같은가

Bitmap

Bitmap은 비트 배열처럼 다루는 자료구조입니다. Redis에서는 문자열을 비트 벡터로 취급하는 비트 연산 집합입니다.

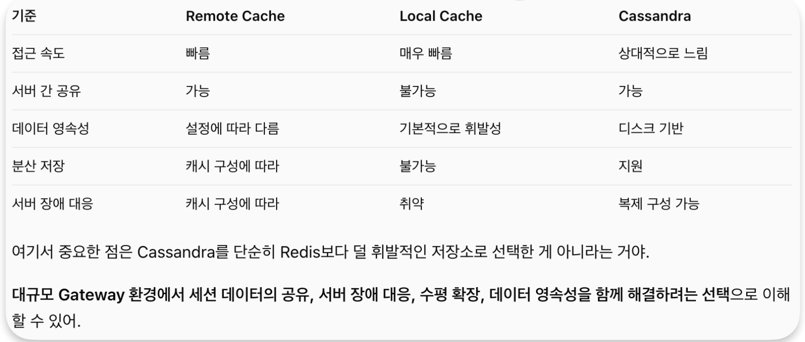
- 예: `user:123`이 오늘 접속했는지 → 비트 1, 아니면 0

Hotkey

Hotkey는 요청이 특정 키에 집중되는 키입니다. 예를 들어 유명인 프로필, 실시간 랭킹 1위, 인기 게시글 같은 경우입니다.

- 문제: 해당 키를 저장한 Redis 노드나 DB 샤드에 트래픽이 몰립니다.
- 해결: 로컬 캐시, 키 복제, 샤딩, 요청 큐잉, CDN 등을 씁니다.

5단계
Gateway + Cassandra



기준	Remote Cache	Local Cache	Cassandra
접근 속도	빠름	매우 빠름	상대적으로 느림
서버 간 공유	가능	불가능	가능
데이터 영속성	설정에 따라 다름	기본적으로 휘발성	디스크 기반
분산 저장	캐시 구성에 따라	불가능	지원
서버 장애 대응	캐시 구성에 따라	취약	복제 구성 가능

여기서 중요한 점은 Cassandra를 단순히 Redis보다 덜 휘발적인 저장소로 선택한 게 아니라는 거야.

대규모 Gateway 환경에서 세션 데이터의 공유, 서버 장애 대응, 수평 확장, 데이터 영속성을 함께 해결하려는 선택으로 이해할 수 있어.

안정적인 구조를 위한 선택

• 구조는 어떻게 진화했는가?

4단계
gateway + local cache + sticky session
각 서버 인스턴스가 자기 메모리 안에 캐시를 두고,
같은 사용자 요청은 같은 서버로 보내 캐시 적중률을 높임

Redis : 인메모리 key-value 데이터 저장소 소프트웨어 자체
└ 운영모드 (Redis Standalone : Redis 인스턴스 1대로 운영하는 가장 단순한 형태
Redis Cluster : 데이터를 여러 master 노드에 자동 분산(sharding)하고, 장애 시 복제본을 승격시키는 분산 Redis 구성
└ 기능 Redis Replication : master 1대 + replica 여러 대. 읽기 복제·장애 대비
└ 별도 컴포넌트 Redis Sentinel : master-replica 구성의 장애 감지하여 자동 페일오버를 도와주는 별도 구성요소

페일오버(Failover)
장애가 난 주 서버(master)를 쓸 수 없게 됐을 때, 대기 중인 복제 서버(replica)를 새 주 서버로 승격시키고 트래픽을 그쪽으로 돌리는 절차

Failback : 원래 master가 복구된 뒤, 다시 원래 구성으로 트래픽을 되돌리는 것
Promotion : replica를 master로 승격시키는 것
Demotion : 기존 master를 replica로 강등하는 것

local cache
각 애플리케이션 인스턴스의 메모리에 두는 캐시.

sticky session
로드밸런서가 같은 사용자의 요청을 계속 같은 백엔드 인스턴스로 보내는 방식. 쿠키/사용자ID 해시/IP 해시 등으로 구현하여 로컬 캐시 히트율을 높임

Cassandra
- 여러 노드에 데이터 복제·분산 저장하는 분산 NoSQL DB
- 세션이나 사용자 상태를 중앙 저장해 특정 서버 인스턴스에 종속되지 않게 함
- 영속적임
- 리더 없는 복제 : 여러 복제 서버에 직접 쓰기를 보내고, 정해진 수의 서버가 쓰기 성공하면 완료된 것으로 판단
- 가십 프로토콜

redis
문자열 뿐만 아니라 Hash, Set 등 다양한 자료구조 제공. 인메모리 데이터 저장소.
캐시뿐 아니라 세션 저장, 랭킹, 큐, pub/sub, 분산 락, 실시간 카운터 등에도 쓰임

Memcached
여러 서버의 RAM을 하나의 논리적 캐시처럼 사용할 수 있는 인프라.
클라이언트가 해시로 노드(서버)를 선택.
기본적으로 서버 간 데이터 복제/전파 없고, 영속화도 X

passport
Netflix에서 도입한 개념.
인증 결과와 사용자/기기 컨텍스트를 하나의 토큰으로 묶어 트랜잭션 안에서 전달하는 객체

gateway
클라이언트와 내부 서비스 사이의 입구. 라우팅, 인증 검사, 속도 제한, TLS 종료, 로깅 등을 맡음

user server
사용자 조회, 프로필, 권한, 세션 검증 등 사용자 도메인 로직을 수행하는 백엔드 서비스

remote cache
애플리케이션 서버 외부의 공유 캐시. ex) redis/Memcached

3단계
gateway + passport
gateway가 요청마다 인증 서버에서 발급받아 downstream 서비스로 전파

2단계
gateway + user server
클라이언트 요청을 먼저 받는 게이트웨이가 라우팅/인증/공통 처리를 하고, 실제 사용자 로직은 user server가 담당

1단계
user server + remote cache
사용자 정보를 처리하는 서버다. 여러 서버가 공유하는 외부 캐시 저장

toss.tech